Instituto Politécnico Nacional
Centro de Investigación en Computación

Desarrollo de Shaders en Godot para Videojuegos

Felipe de Jesús Martínez Alfaro



Congreso CORE B26

Contenido

Capítulo 1. 3
Conoce Godot. 3
¿Qué es un … *Shader*? 4
Funciones *Shader* en Godot. 4
Vertex. 4
Fragment. 4
Light. 4
Start. 4
Process. 5
Sky. 5
Fog. 5
Tipos de *Shaders* en Godot. 5
Render Pipeline. 6
Fotograma. 6
Procesamiento de Físicas y de nodos (Idle and Physics Processing). 6
Práctica: Simular la gravedad entre cuerpos celestes. 7
Descargar Godot. 7
Crea tus escenas 8
Crea los programas. 9
Reto 9
Diagrama de comportamiento. 10

Capítulo 1.

Conoce Godot.

Inicialmente desarrollado en Argentina por Ariel Manzur y Juan Linietsky en los 2000s, Godot es un motor de videojuegos gratuito, de código abierto y no requiere registro para poder descargarlo.

Actualmente cuenta con más de 3,000 desarrolladores, así como varios patrocinadores entre ellos SCOREWARRIOR, BATTLEFIELD STUDIOS, W4 Games, etc.

Algunos proyectos desarrollados en Godot son:

- PVKK: Planetenverteidigungskanonenkommandant



PVKK: Planetenverteidigungskanonenkommandant en Steam

- Zitifono



Zitifono en Steam

ENLACES:
Historia: A decade in retrospective and future – Godot Engine
Historia: Godot history in images! – Godot Engine
Página: Godot Engine - Motor de videojuegos 2D y 3D gratuito y de código abierto
Codigo fuente: godotengine/godot: Godot Engine – Multi-platform 2D and 3D game engine

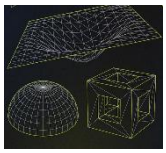
¿Qué es un … *Shader*?

Un *Shader* es un script que se ejecuta en la GPU dentro de un proceso de renderizado, cuya función es controlar cómo se procesa la información gráfica para generar o modificar la apariencia visual de una escena. A través de operaciones matemáticas y del procesamiento de datos como posiciones, coordenadas, colores, normales y texturas, un *Shader* puede determinar cómo se dibujan los objetos, cómo interactúan con la luz o cómo se modifican sus píxeles para producir efectos visuales.

En términos coloquiales, un *Shader* es un pequeño programa que le indica a la GPU cómo dibujar algo o cómo transformar su apariencia visual. Por ejemplo, puede hacer que un personaje tenga un contorno, simular agua en movimiento, generar distorsiones, aplicar iluminación o crear efectos de disolución. Aunque suele asociarse con los materiales y los efectos visuales, su función depende de la etapa del renderizado en la que se ejecuta y del tipo de *Shader* utilizado.

Funciones *Shader* en Godot.

Vertex.



La función vertex() se ejecuta sobre todos los vértices de la malla y establece sus posiciones y otras variables asociadas a cada vértice. Se utiliza en shaders canvas_item y spatial.

Ejemplo: Desplazar los vértices de un objeto para deformarlo o hacer que una superficie parezca ondularse.

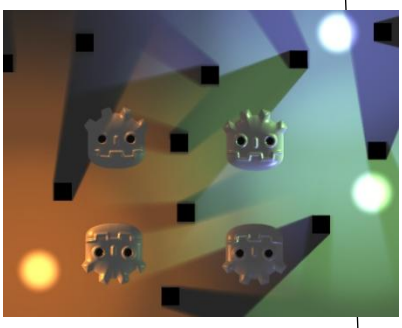
Fragment.



La función fragment() se ejecuta para cada fragmento cubierto por la malla. Utiliza valores generados por vertex(), interpolados entre los vértices. Se utiliza en shaders **canvas_item y spatial**.

Ejemplo: Cambiar el color de cada píxel de un sprite para crear un efecto de degradado.

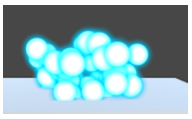
Light.



La función light() se ejecuta para cada fragmento y para cada fuente de luz. Utiliza variables de fragment() y de sus ejecuciones anteriores. Se utiliza en shaders canvas_item y spatial.

Ejemplo: Modificar cómo una luz ilumina un objeto para obtener un efecto de iluminación personalizado.

Start.



La función start() se ejecuta una vez para cada partícula de un sistema cuando esta aparece por primera vez. Se utiliza en shaders de partículas.

Ejemplo: Asignar una velocidad inicial aleatoria a cada partícula de una explosión.

ENLACES:
Documentación: Introducción a los shaders — Documentación de Godot Engine (4.x) en español

Process.

La función process() se ejecuta para cada partícula de un sistema en cada fotograma. Se utiliza en shaders de partículas.



Ejemplo: Modificar la velocidad de las partículas en cada fotograma para simular la gravedad.

Sky.



La función sky() se ejecuta para cada píxel del mapa cúbico de radiancia cuando este necesita actualizarse y para cada píxel de la pantalla actual. Se utiliza en shaders de cielo.

Ejemplo: Generar un cielo con estrellas y nebulosas mediante cálculos matemáticos, sin utilizar una imagen de fondo.

Fog.

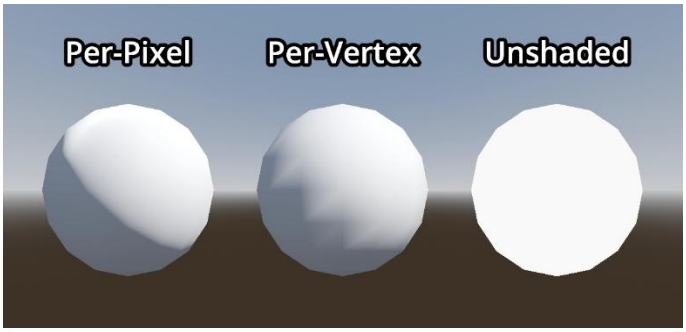
La función fog() se ejecuta para cada froxel del búfer de niebla volumétrica que intersecta con un nodo FogVolume. Un froxel es una celda de una representación tridimensional discretizada del espacio. Se utiliza en shaders de niebla.



Ejemplo: Crear una zona de niebla coloreada dentro de un volumen para simular humo o una atmósfera fantástica.

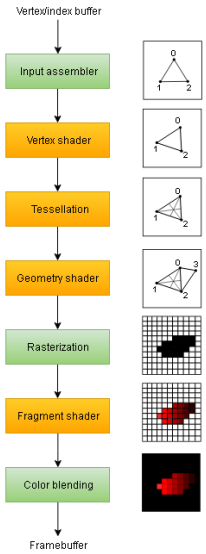
Tipos de *Shaders* en Godot.

- spatial para renderizado 3D.
- canvas_item para renderizado 2D.
- particle para sistemas de partículas.
- sky para renderizar materiales Sky o cielos.
- fog para renderizar nieblas de volumen.



Render Pipeline.

El render pipeline (tubería de renderizado) es la secuencia ordenada de etapas por las que pasa la información gráfica para transformarse en una imagen que se muestra en pantalla. Durante este proceso, la GPU y otros componentes del sistema realizan tareas como procesar la geometría, calcular colores, aplicar iluminación y combinar los resultados para generar la imagen final. Los shaders son pequeños programas que intervienen en determinadas etapas de este proceso para controlar cómo se calcula o modifica el resultado visual.



Fotograma.

Un **frame o fotograma** es una imagen individual que forma parte de una secuencia de imágenes que, al mostrarse rápidamente una tras otra, producen la sensación de movimiento.

En videojuegos, cada fotograma representa el resultado de un instante del juego. La computadora procesa la escena, ejecuta las operaciones necesarias del render pipeline y genera la imagen que se muestra en pantalla.

Procesamiento de Físicas y de nodos (Idle and Physics Processing).

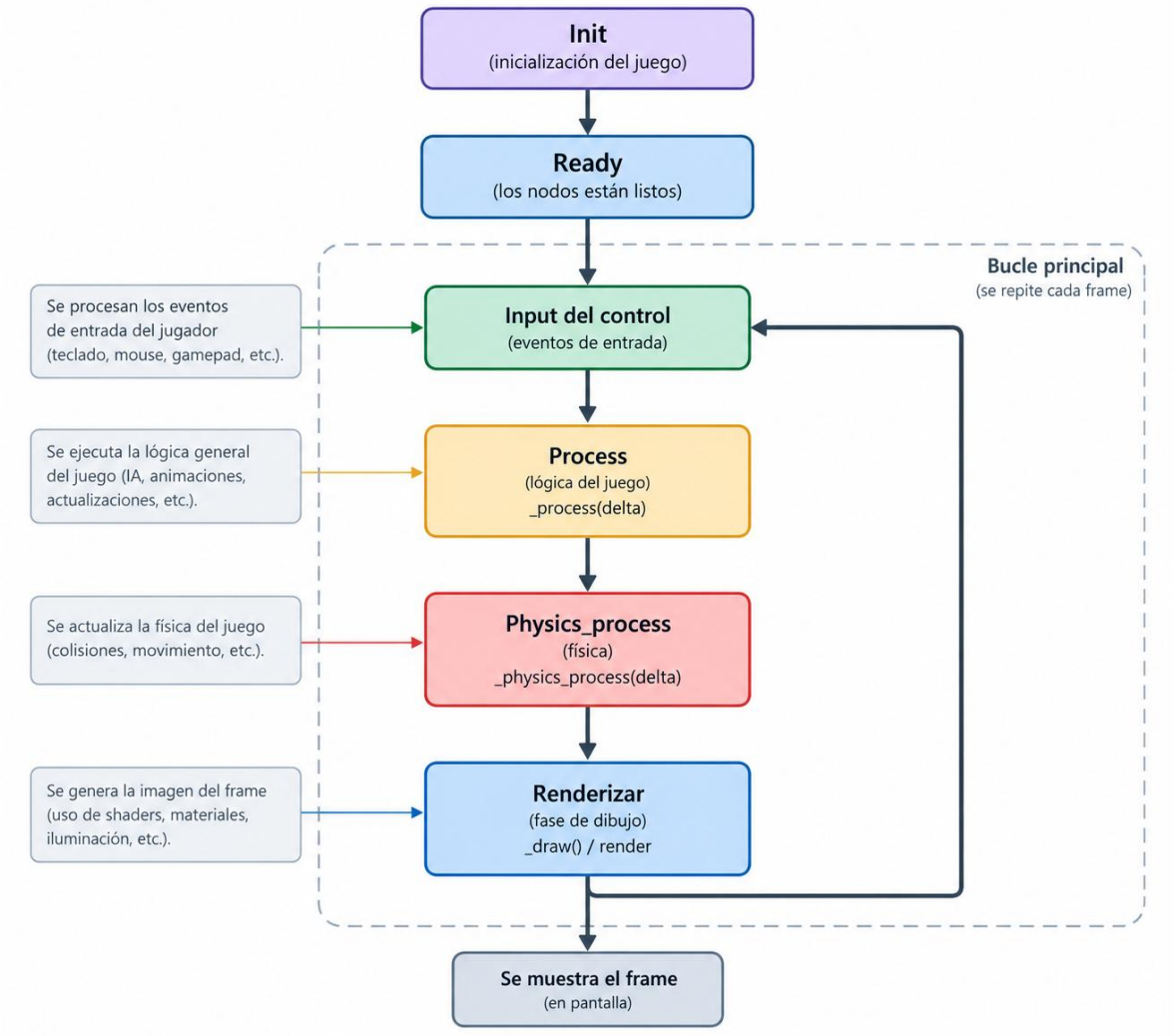
En cada fotograma, es necesario actualizar el estado del mundo del juego antes de dibujarlo en pantalla. Godot ofrece dos métodos virtuales en la clase **Node** para realizar esta tarea: Node._process() y Node._physics_process()`. Si defines uno o ambos en un script, el motor los llamará automáticamente.

Existen dos tipos de procesamiento a tu disposición:

- El procesamiento en reposo (*idle processing*) te permite ejecutar código que actualiza un nodo en cada fotograma, con la mayor frecuencia posible.
- El procesamiento físico (*physics processing*) ocurre a una frecuencia fija, por defecto 60 veces por segundo. Este proceso es independiente de la tasa de fotogramas real de tu juego y garantiza que las físicas funcionen de manera fluida. Debes utilizarlo para cualquier elemento que involucre al motor de físicas, como el movimiento de un cuerpo que colisiona con el entorno.

ENLACES:
Documentación:

Idle and Physics Processing — Godot Engine (stable) documentation in English



Práctica: Simular la gravedad entre cuerpos celestes.

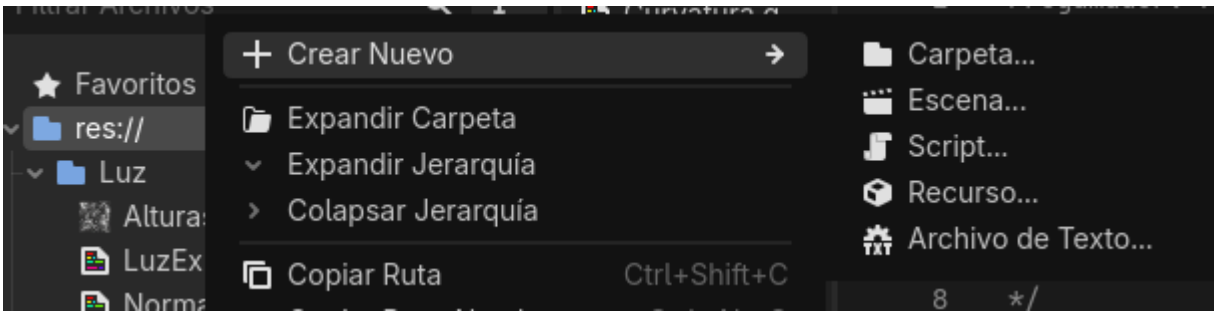
Descargar Godot.

Para empezar nuestros proyectos debemos empezar con el motor de videojuegos de Godot, yendo a la página oficial. [Godot Engine - Motor de videojuegos 2D y 3D gratuito y de código abierto]

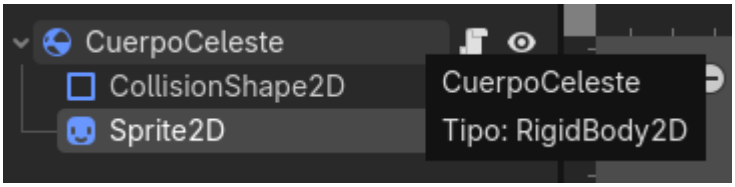


Crea tus escenas

Una vez abierto el programa daremos a crear un nuevo proyecto, cuando nos cargue en la sección de carpetas haremos clic derecho para crear las instancias necesarias.

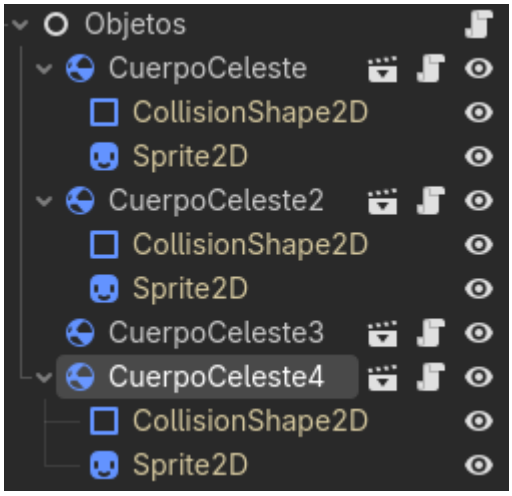


- Crea una carpeta para nuestro proyecto de planetas.
- Crea una escena (Yo la llamé planetario).
- A la escena haz clic derecho en el nodo raíz y crea un objeto RigidBody2D con las siguientes características:

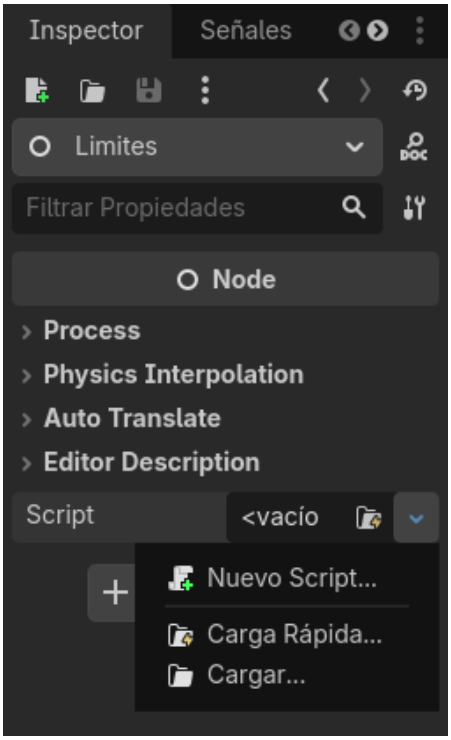


Una vez configurado nuestro nodo, podemos guardarlo como una escena nueva dando clic derecho sobre la raíz de nuestro nodo, yo la llamé “Cuerpo celeste”.

- Crea un “Node” simple.
- Agrégale varias instancias de “Cuerpo celeste” como hijo de ese nodo simple.



Ahora podemos agregarle un script al nodo objetos y al de la escena cuerpo celeste para que se comporten con la lógica esperada en gdscript.



Al dar clic en el nodo, se nos seleccionarán sus características en la sección de **Inspector**, donde podremos agregar nuestros programas.

Crea los programas.

Para los cuerpos celestes:

- Solo agrega la posibilidad de asignar una fuerza inicial en la función "_ready()" usando "self.apply_impulse".
- Recuerda usar @export para poder modificar tus variables en el inspector.
- Recuerda en el inspector del RigidBody desactivar la gravedad.

Para el nodo "Objetos" :

Crea una lista de objetos RigidBody2D para ser llenada en el inspector.

Implementa una función para calcular la gravedad.

Implementa una función para sumar la gravedad de los demás cuerpos que le influyen a un nodo.

En el proceso de físicas ("_physics_process") aplica la gravedad a cada objeto de la lista creada.

Finalmente selecciona cada nodo "Cuerpo celeste" y arrástralo a la lista en el inspector del nodo "Objetos".

Reto

¿Como sería una implementación donde cada nodo calcule la suma de sus influencias? Es decir que se calcule dentro del script de "Cuerpo celeste". ¿Tendría una ventaja hacerlo así?.

Diagrama de comportamiento.

A continuación, se muestra un diagrama de como se debe comportar la lógica de nuestra escena.

